TRƯỜNG ĐẠI HỌC BÁCH KHOA – ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

ĐỀ TÀI

Lập trình Bất đồng bộ với Future & Stream trong Flutter

Sinh viên thực hiện: **Phan Văn Trà**
Trần Đăng Duy

Giảng viên hướng dẫn: **TS. Nguyễn Duy Nhật Viễn**

Đà Nẵng, ngày 26 tháng 11 năm 2026

Mục lục

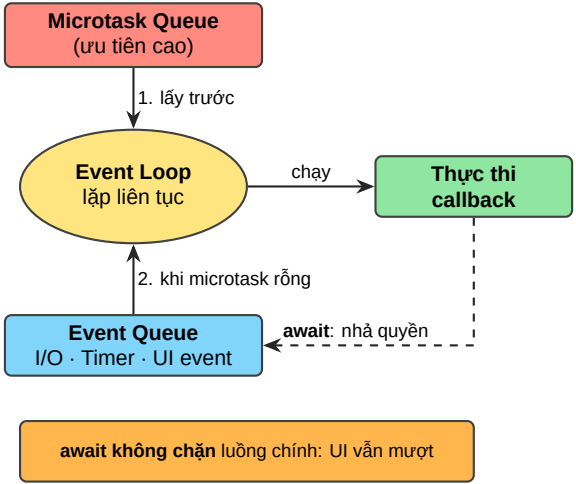
1. Cơ sở lý thuyết
1.1 Cơ chế bất đồng bộ: Future & async/await
1.2 Luồng dữ liệu liên tục: Stream
1.3 Cơ chế dọn dẹp bộ nhớ

2. Giới thiệu project FocusFlow (Pomodoro)
2.1 Tổng quan project
2.2 Áp dụng Future & Stream và luồng hoạt động

3. Kết luận

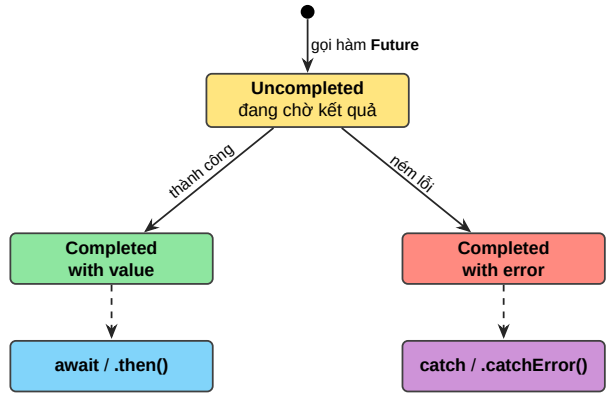
1.1 Vì sao cần lập trình bất đồng bộ?

- Dart chạy trên **một luồng (single-thread)** cho mỗi **isolate**.
- Tác vụ chờ: **mạng, đọc/ghi file, timer, cơ sở dữ liệu** … nếu chặn luồng chính ⇒ **UI bị đứng (jank)**.
- **Event Loop** lần lượt lấy sự kiện từ hai hàng đợi:
  1. **Microtask Queue** (ưu tiên cao)
  2. **Event Queue** (I/O, Timer, UI event)
- **Future** và **Stream** là hai công cụ cốt lõi của Dart để làm việc với Event Loop.



1.1 Future: đại diện cho một kết quả “sẽ có trong tương lai”

- **`Future<T>`** biểu diễn **một giá trị duy nhất** (hoặc một **lỗi**) được trả về **sau**.
- Có **3 trạng thái**: **Uncompleted** → **Completed with value** hoặc **Completed with error**.
- Nhận kết quả bằng:
  - **`.then()`** / **`.catchError()`** / **`.whenComplete()`**
  - **`async/await`** + **`try/catch/finally`**
- Tạo Future: **`Future(...)`**, **`Future.delayed()`**, **`Future.value()`**, **`Future.error()`**.



1.1 Cú pháp async/await: viết bất đồng bộ như đồng bộ

Cách 1: `then` / `catchError`

```
Future<String> fetchUser() {
  return Future.delayed(
    Duration(seconds: 2),
    () => 'Nguyen Van A',
  );
}

void main() {
  fetchUser()
    .then((name) => print('Xin chao $name'))
    .catchError((e) => print('Loi: $e'))
    .whenComplete(() => print('Xong'));
}
```

Cách 2: `async` / `await` (khuyên dùng)

```
Future<String> fetchUser() async {
  await Future.delayed(Duration(seconds: 2));
  return 'Nguyen Van A';
}

Future<void> main() async {
  try {
    final name = await fetchUser();
    print('Xin chao $name');
  } catch (e) {
    print('Loi: $e');
  } finally {
    print('Xong');
  }
}
```

- Hàm **async** luôn trả về **`Future`**; **await** chỉ dùng được **bên trong** hàm **async**.
- Code dễ đọc hơn, xử lý lỗi bằng **try/catch** quen thuộc, tránh “callback hell”.

1.1 Kết hợp nhiều Future & xử lý lỗi

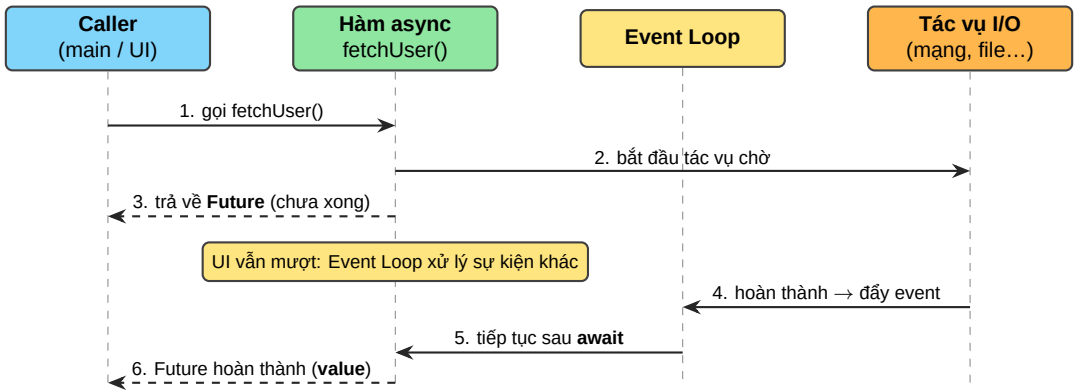
API	Ý nghĩa
`Future.wait([...])`	Chạy **song song**, đợi **tất cả** hoàn thành
`Future.any([...])`	Lấy kết quả của Future **xong đầu tiên**
`.timeout(d)`	Ném **TimeoutException** nếu quá thời gian
`.whenComplete()`	Luôn chạy (giống **finally**)
`Future.delayed()`	Trì hoãn rồi mới chạy

```dart
// Chay song song 2 tac vu
final results = await Future.wait([
  loadProfile(),
  loadSettings(),
]);

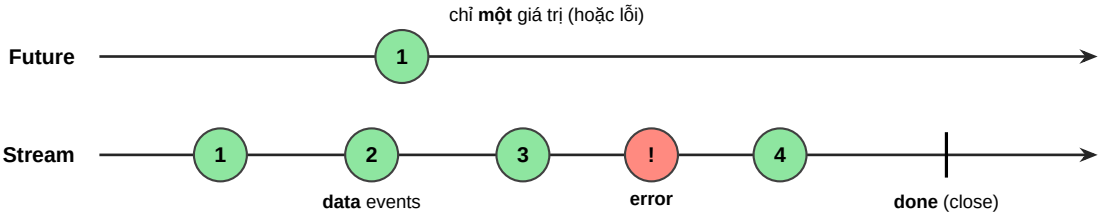
// Gioi han thoi gian cho
try {
  final data = await fetchData()
      .timeout(Duration(seconds: 5));
} on TimeoutException {
  print('Qua thoi gian cho!');
} catch (e) {
  print('Loi khac: $e');
}
```

Lưu ý: luôn bắt lỗi (**try/catch**) quanh **await**, nếu không lỗi sẽ thành **unhandled exception**.

1.1 Sơ đồ tuần tự: luồng chạy của async/await

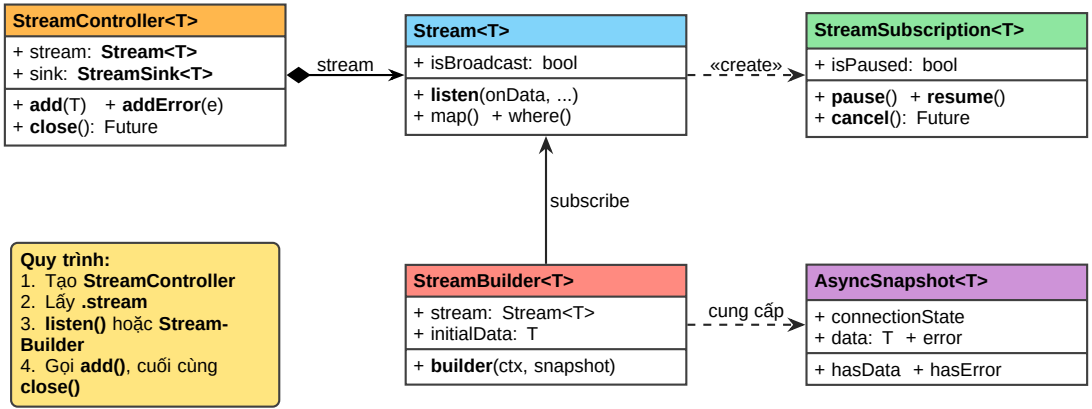


1.2 Stream: chuỗi sự kiện bất đồng bộ theo thời gian



Tiêu chí	Future	Stream
Số giá trị	**Một** giá trị (hoặc lỗi)	**Nhiều** giá trị theo thời gian
Kết thúc	Tự kết thúc sau khi có kết quả	Kết thúc khi gọi `close()`
Cách nhận	`await`, `then`	`listen`, `await for`, `StreamBuilder`
Widget Flutter	`FutureBuilder`	`StreamBuilder`
Loại	—	**Single-subscription** (1 listener) hoặc **Broadcast** (nhiều listener)

1.2 Sơ đồ lớp UML: các thành phần của Stream



1.2 StreamController: phát dữ liệu & lắng nghe sự kiện

```dart
final controller = StreamController<int>();

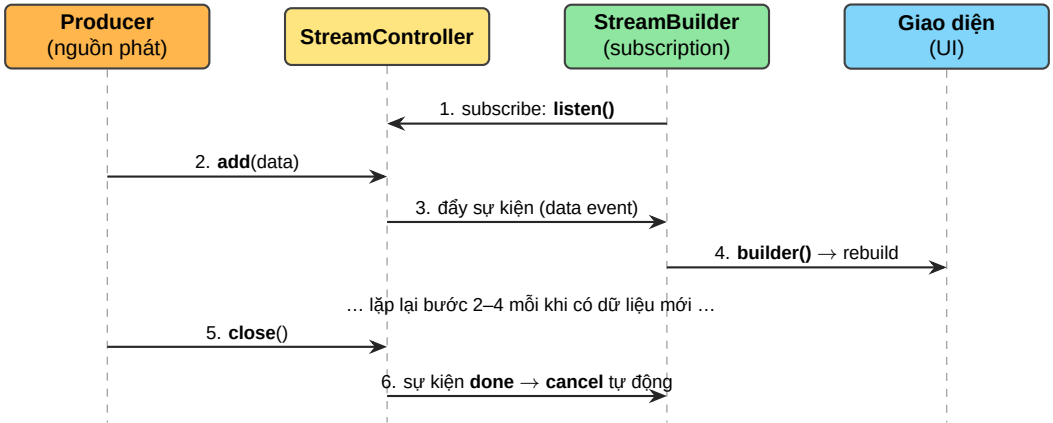
// Consumer: lang nghe (listener)
final sub = controller.stream.listen(
  (data) => print('Nhan: $data'),
  onError: (e) => print('Loi: $e'),
  onDone: () => print('Ket thuc'),
);

// Producer: phat du lieu
controller.add(1);
controller.add(2);
controller.addError(Exception('oops'));
controller.close();   // bao hieu "done"

// Hoac tao Stream bang async*
Stream<int> countDown(int n) async* {
  for (var i = n; i >= 0; i--) {
    await Future.delayed(Duration(seconds: 1));
    yield i;       // phat 1 gia tri
  }
}
```

- **`StreamController`** là **nguồn phát**: **`add()`** gửi dữ liệu, **`addError()`** gửi lỗi, **`close()`** kết thúc.
- **`controller.stream`** là **đầu ra** để các listener đăng ký.
- **`listen()`** trả về **`StreamSubscription`** để **pause / resume / cancel**.
- **Single-subscription**: chỉ 1 listener (mặc định).
  Broadcast: **`StreamController.broadcast()`** cho **nhiều** listener.
- **`async*`** + **`yield`**: cách **ngắn gọn** để tạo Stream.

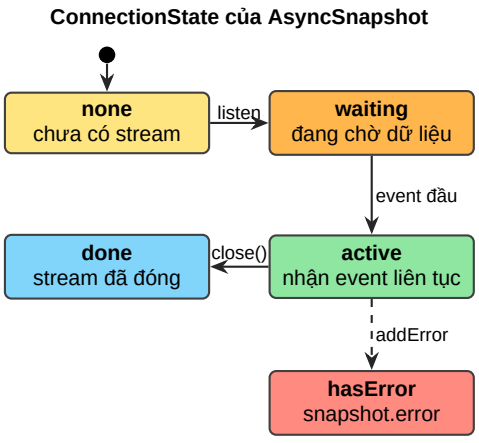
1.2 Sơ đồ tuần tự: phát dữ liệu & cập nhật giao diện



1.2 StreamBuilder: cập nhật giao diện theo Stream

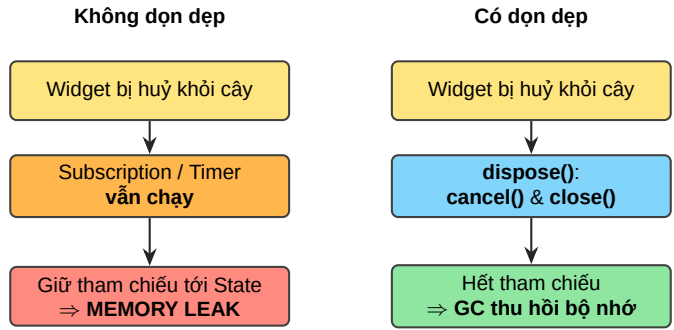
```
StreamBuilder<int>(
  stream: controller.stream,
  initialData: 0,
  builder: (context, snapshot) {
    if (snapshot.hasError) {
      return Text('Loi: ${snapshot.error}');
    }
    if (snapshot.connectionState ==
        ConnectionState.waiting) {
      return CircularProgressIndicator();
    }
    return Text('Gia tri: ${snapshot.data}');
  },
)
```

- **StreamBuilder** tự **subscribe** khi gắn vào cây widget và tự **cancel** khi bị huỷ.
- Mỗi sự kiện mới ⇒ gọi lại **`builder`** ⇒ chỉ **rebuild** phần giao diện liên quan.



1.3 Rò rỉ bộ nhớ (memory leak) khi quên cancel / dispose

- **Subscription** và **Timer** giữ tham chiếu tới **State/callback**.
- Widget bị huỷ mà **không cancel** ⇒ đối tượng **không được GC thu hồi**.
- Hậu quả: **memory leak**, tốn CPU/pin, lỗi **`setState() called after dispose()`**.
- Giải pháp: **huỷ mọi thứ đã đăng ký** trong **`dispose()`**.



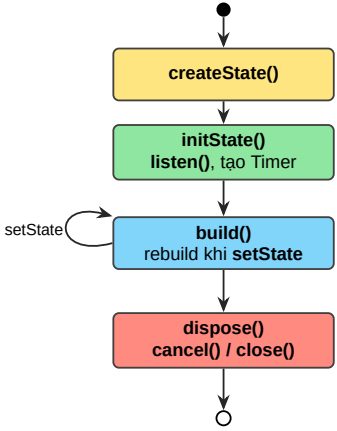
Checklist dọn dẹp: `StreamSubscription.cancel()` · `Timer.cancel()` · `StreamController.close()` · kiểm tra `mounted` trước khi gọi `setState()`

1.3 Vòng đời State & nơi dọn dẹp

```dart
class _MyState extends State<MyWidget> {
  late StreamSubscription<int> _sub;
  final _controller = StreamController<int>();
  Timer? _timer;

  @override
  void initState() {
    super.initState();
    _sub = _controller.stream.listen(
      (v) => setState(() => _value = v));
    _timer = Timer.periodic(
      Duration(seconds: 1), (_) => _tick());
  }

  @override
  void dispose() {
    _sub.cancel();          // huy lang nghe
    _timer?.cancel();       // dung timer
    _controller.close();    // dong nguon phat
    super.dispose();
  }
}
```



1.3 Tóm tắt phần cơ sở lý thuyết

Future

- **Một** kết quả trong tương lai
- **async/await**, **try/catch**
- **`Future.wait`**, **`timeout`**
- Dùng cho: gọi API, đọc/ghi dữ liệu, lưu lịch sử

Stream

- **Nhiều** sự kiện theo thời gian
- **StreamController**: **`add`**, **`close`**
- **listen / StreamBuilder**
- Dùng cho: đồng hồ đếm, chat, cảm biến

Dọn dẹp

- **cancel()** subscription
- **close()** controller, **cancel()** Timer
- Thực hiện trong **dispose()**
- Tránh **memory leak** & lỗi sau khi widget bị huỷ

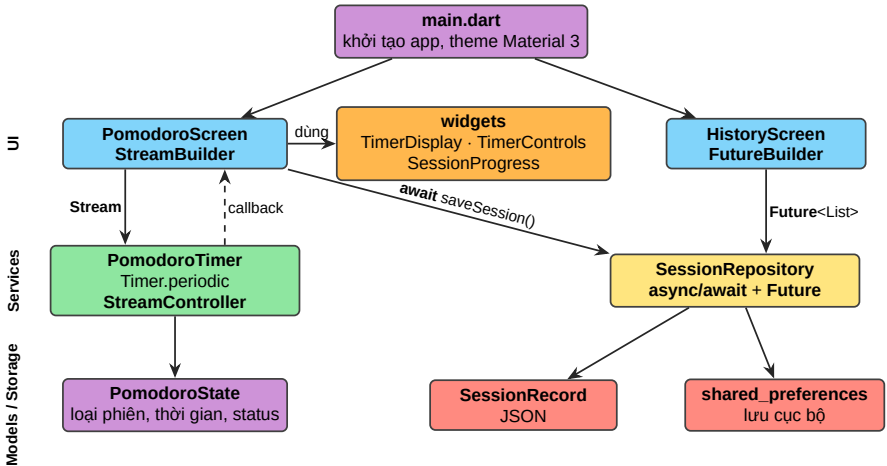
2.1 FocusFlow – ứng dụng đồng hồ Pomodoro

- Ứng dụng **Pomodoro Timer** cho **Android**, xây dựng bằng **Flutter/Dart**, **hoạt động offline**.
- **3 chế độ**: Tập trung (**25'**), Nghỉ ngắn (**5'**), Nghỉ dài (**15'**).
- Điều khiển: **Start, Pause, Resume, Reset**.
- **Tự động chuyển phiên**; **4 phiên tập trung** → nghỉ dài.
- Tuỳ chỉnh thời gian tập trung **1–180 phút**, lưu trên thiết bị.
- **Lịch sử phiên** đã hoàn thành.
- Giao diện **Material 3**: xanh lá = tập trung, cam = nghỉ.

Thành phần	Chi tiết
Ngôn ngữ	Dart 3
Framework	Flutter 3 (Material 3)
Bất đồng bộ	`Future` & `Stream` / `StreamController`
Lưu trữ	`shared_preferences`
State	`StreamBuilder`
Kiểm thử	`flutter_test`, `fake_async`

GitHub: `https://github.com/Duy-OC/asynchronous_futures_streams`

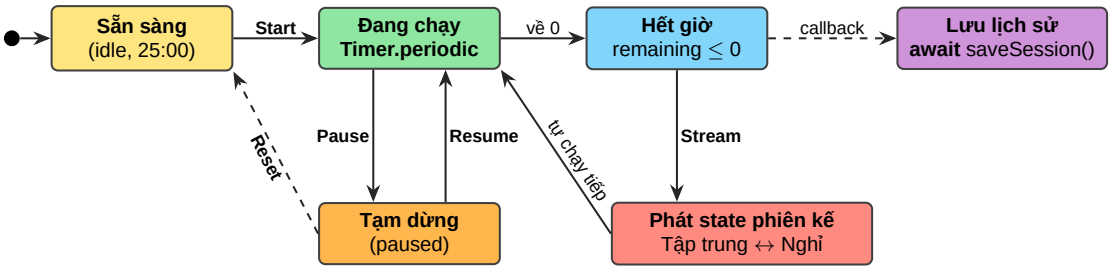
2.1 Kiến trúc project (theo cấu trúc thư mục `lib/`)



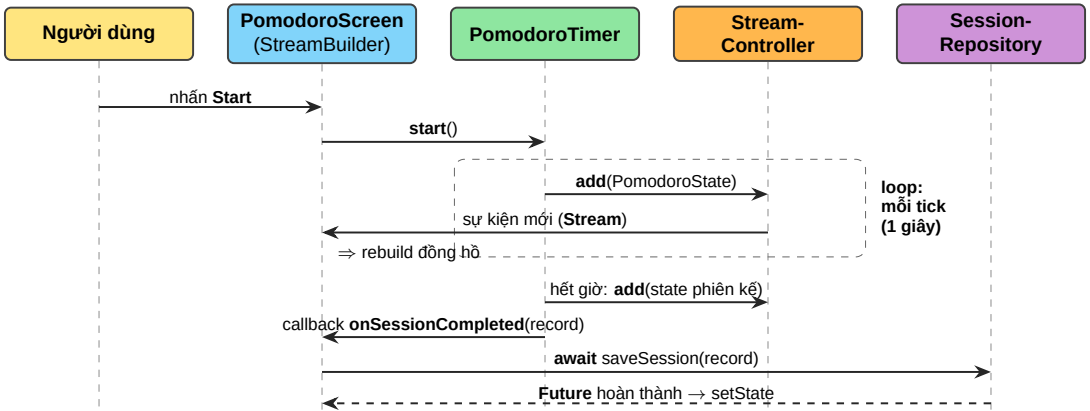
2.2 Áp dụng Future & Stream trong project

Thành phần	Kỹ thuật bất đồng bộ	Vai trò
PomodoroTimer	**`Timer.periodic`** + **`StreamController.broadcast()`**	Mỗi **tick (1 giây)** phát **PomodoroState** mới qua **Stream**
PomodoroScreen	**`StreamBuilder`** + **`Future`**	**Lắng nghe Stream** để cập nhật UI; callback **`onSessionCompleted`** xếp hàng lưu bằng **`_pendingSave.then()`**
SessionRepository	**`async/await`** + **`Future`**	**`loadSessions()`** / **`saveSession()`** đọc/ghi **`shared_preferences`** **không chặn UI**
HistoryScreen	**`FutureBuilder`**	**Một lần đọc**: đang tải / lỗi / có kết quả
Dọn dẹp	**`_ticker?.cancel()`**, **`_stateController.close()`**, **`dispose()`**	**Dừng Timer, đóng Stream** khi rời màn hình ⇒ tránh **memory leak**
Kiểm thử	**`fake_async`**	Kiểm thử đếm ngược **không cần chờ thời gian thật**

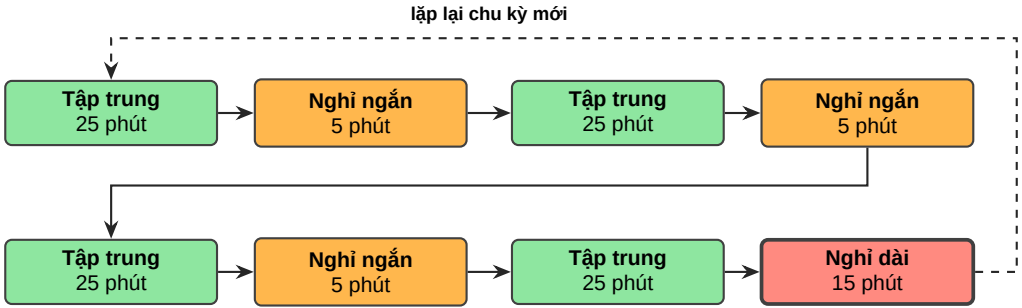
2.2 Sơ đồ trạng thái: luồng hoạt động của đồng hồ



2.2 Sơ đồ tuần tự: Stream cập nhật UI & Future lưu lịch sử



2.2 Chu kỳ phiên Pomodoro



- Ứng dụng đếm **số phiên tập trung** trong chu kỳ; đủ **4 phiên** thì chuyển sang **nghỉ dài**.
- Mỗi lần hết giờ: **Stream** phát state của phiên kế tiếp (**tự động chạy**) và gọi callback → **Future** lưu lịch sử.

2.2 Code thực tế (1/3): Stream – `pomodoro_timer.dart`

```dart
// pomodoro_timer.dart
// STREAM: phát toàn bộ trạng thái đồng hồ cho giao diện.
final StreamController<PomodoroState> _stateController =
    StreamController<PomodoroState>.broadcast();
Stream<PomodoroState> get stream => _stateController.stream;

void _startTicker() {
  _endTime = _now().add(_state.remaining);
  _ticker?.cancel();
  _ticker = Timer.periodic(tickInterval, (_) => _onTick());
  _emit(_state.copyWith(status: TimerStatus.running));
}

void _emit(PomodoroState nextState) {
  if (_disposed) return;
  _state = nextState;
  _stateController.add(nextState);   // phát cho listener
}

// Thoát màn hình: hủy nguồn phát trước, rồi đóng Stream.
void dispose() {
  if (_disposed) return;
  _disposed = true;
  _stopTicker();                     // _ticker?.cancel()
  unawaited(_stateController.close());
}
```

Phát dữ liệu
`Timer.periodic` gọi **`_onTick()`** mỗi giây; **`_emit()`** gọi **`add()`** để phát **PomodoroState**.

Broadcast
`StreamController.broadcast()` cho phép **nhiều listener** đăng ký.

Dọn dẹp
`dispose()`: **cancel** Timer, rồi **close** Stream.

2.2 Code thực tế (2/3): StreamBuilder + Future

```dart
// pomodoro_screen.dart
_timer = PomodoroTimer(onSessionCompleted: _saveCompleted);
// Xếp các lần lưu nối tiếp để hai phiên không ghi đè nhau.
void _saveCompleted(SessionRecord record) {
  _pendingSave =
      _pendingSave.then((_) => _persistSession(record));
}
Future<void> _persistSession(SessionRecord record) async {
  try {
    await _repository.saveSession(record);      // FUTURE
    if (!mounted) return;
    setState(() { /* cập nhật số phiên đã lưu */ });
  } catch (_) { /* hiển thị lỗi lưu */ }
}
void dispose() {
  _timer.dispose();
  super.dispose();
}
// StreamBuilder tự đăng ký/hủy lắng nghe
body: StreamBuilder<PomodoroState>(
  stream: _timer.stream,
  initialData: _timer.state,
  builder: (context, snapshot) { ... },
)
```

Stream → UI
`StreamBuilder` nhận state mới và vẽ lại đồng hồ.

Future
`await saveSession()` lưu lịch sử mà **không chặn UI**; **`mounted`** kiểm tra trước **`setState`**.

Dọn dẹp
`dispose()` của State gọi **`_timer.dispose()`**.

2.2 Code thực tế (3/3): Future & FutureBuilder

```dart
// session_repository.dart
Future<List<SessionRecord>> loadSessions() async {
  final preferences = await _preferencesProvider();
  final raw = preferences.getString(storageKey);
  if (raw == null || raw.isEmpty) return <SessionRecord>[];
  ...   // jsonDecode -> danh sách SessionRecord
}

Future<void> saveSession(SessionRecord session) async {
  final sessions = await loadSessions();
  sessions.add(session);
  await saveSessions(sessions);
}

// history_screen.dart
// Tạo Future một lần, không đọc bộ nhớ lại sau mỗi lần build.
_sessions = widget.repository.loadSessions();   // initState

body: FutureBuilder<List<SessionRecord>>(
  future: _sessions,
  builder: (context, snapshot) {
    if (snapshot.connectionState != ConnectionState.done) {
      return const Center(child: CircularProgressIndicator());
    }
    ...
```

async/await
Đọc/ghi **`shared_preferences`** bằng **`await`**, trả về **`Future`**.

FutureBuilder
Một lần đọc: đang tải → lỗi / kết quả → hiển thị danh sách.

Link GitHub
`github.com/Duy-OC/asynchronous_futures_streams`

3. Kết luận

- **Future** + **async/await**: xử lý tác vụ chờ cho **một kết quả** mà không chặn UI.
- **Stream** + **StreamController** + **StreamBuilder**: phát, lắng nghe và cập nhật giao diện theo **luồng dữ liệu liên tục**.
- **cancel() / close() / dispose()**: bắt buộc để tránh **memory leak**.
- Project **FocusFlow (Pomodoro)** áp dụng đủ ba kỹ thuật: **Stream** cho đồng hồ, **Future** cho lịch sử, **dispose** để dọn dẹp.

Xin cảm ơn
thầy và các bạn đã lắng nghe!

Hỏi & Đáp